DSA — MODULE 4

Separate colourful notes • Trees, balanced search trees, expression trees & hashing

Prepared from your uploaded Module 4 lecture slides and PDFs. Sorting algorithms are not included here; they remain in their own separate PDF.

> Main coverage: AVL trees • Splay trees • B-trees • Red-black trees • General trees & BST • Binary expression trees • Hashing techniques.

How to write a strong exam answer

Use this order for a long answer: (1) definition and purpose, (2) properties/rules, (3) labelled example or rotation diagram, (4) operation steps, (5) complexity, and (6) one advantage/application. For tree questions, draw the tree after every important rotation or adjustment.

1. Quick tree foundations

A tree is a non-linear hierarchical data structure made of nodes connected by edges. The uploaded tree terminology slides define the root as the first node, edges as links, siblings as nodes with the same parent, leaves as nodes without children, and internal nodes as nodes with at least one child. Depth counts edges from the root; height counts edges on the longest path from a node to a leaf. ■filecite■turn12file3■L71-L75■

Term	Meaning
Root	The top node; has no parent.
Parent / child	A parent directly connects to one or more children.
Sibling	Nodes sharing the same parent.
Leaf / external node	Node with no children.
Internal node	Node with at least one child.
Degree of a node	Number of children.
Depth	Number of edges from root to the node.
Height	Longest downward path, counted in edges.
Subtree	A node together with its descendants.

Binary tree vs Binary Search Tree

A binary tree permits at most two children per node (left and right). A Binary Search Tree (BST) adds an ordering rule: keys in the left subtree are smaller and keys in the right subtree are larger, as described in the supplied BST slides. ■filecite■turn12file3■L359-L374■

Structure	Main rule	Why it matters
Binary tree	At most two children per node.	Represents hierarchical structure.
BST	Left keys < node key < right keys (distinct-key convention).	Search follows comparisons.
Full binary tree	Each node has 0 or 2 children.	No node has exactly one child.
Complete binary tree	All levels filled except possibly bottom; bottom fills left-to-right.	Useful for array-based heaps.

2. AVL Trees

An AVL tree is a height-balanced Binary Search Tree. The uploaded AVL slides define a node's balance factor as height(left subtree) – height(right subtree); each node must have balance factor –1, 0 or +1. If insertion makes a node's balance factor +2 or –2, rotations restore the AVL property. ■filecite■turn12file0■L184-L189■

Balance factor

```
BF(node) = height(left subtree) - height(right subtree)

AVL-valid balance factors: -1, 0, +1
BF = +2 or -2 after an update -> rebalance is required.
```

Four imbalance cases

Case	Where the new key is inserted	Fix
LL	Left subtree of left child	Single right rotation
RR	Right subtree of right child	Single left rotation
LR	Right subtree of left child	Left rotation on child, then right rotation
RL	Left subtree of right child	Right rotation on child, then left rotation

How to identify the case

Start from the first unbalanced ancestor and follow the path from that ancestor to the inserted key. Two left moves indicate LL; two right moves indicate RR; left then right indicates LR; right then left indicates RL.

Rotation sketches

```
LL:                 Right rotate at z:
        z                     y
       /                     / \
      y          ->         x   z
     /
    x

RR is the mirror image: left rotate at z.

LR: left-rotate y, then right-rotate z.
RL: right-rotate y, then left-rotate z.
```

AVL insertion procedure

1. Insert the key using ordinary BST insertion.
2. Move upward from the inserted node toward the root.
3. Update heights and calculate each balance factor.
4. At the first node with BF +2 or –2, identify LL, RR, LR or RL.
5. Apply the corresponding rotation(s), then check the resulting subtree.

Operation	Time complexity (height-balanced AVL)
Search	O(log n)
Insertion	O(log n)
Deletion	O(log n)

> Exam tip: Always state the BF formula explicitly. Do not mix it with the opposite convention height(right) – height(left); if using that convention, the signs reverse.

3. Splay Trees

A splay tree is a self-adjusting BST. After a node is accessed, splaying applies rotations to bring that node to the root. The supplied lecture notes emphasize locality: recently accessed items tend to stay near the root. ■filecite■turn12file1■L64-L70■

Three splaying patterns

Pattern	Relationship of X, parent P, grandparent G	Action
Zig	X's parent P is the root; no grandparent	One rotation.
Zig-Zig	P and X are on the same side of G (left-left or right-right)	Two rotations in the same direction.
Zig-Zag	P and X are on opposite sides (left-right or right-left)	Two rotations in opposite directions.

Operations

Find: perform ordinary BST search, then splay the accessed node to the root. **Insert:** insert as in a BST, then splay the new node. **Delete:** splay the target to the root, remove it, then join the left and right subtrees; the supplied lecture describes splaying the maximum node of the left subtree and attaching the right subtree. ■filecite■turn12file1■L299-L305■

Property	Exam-ready point
Strict balance?	No; the tree is not perfectly balanced at every moment.
Why useful?	Frequently accessed nodes move toward the root.
Amortized performance	O(log n) per operation over a sequence of operations, as described in the lecture.
Worst case for one operation	Can be O(n) for an individual operation.

> Compare AVL vs splay: AVL explicitly maintains a height-balance condition; splay trees adjust after access and rely on amortized performance.

4. B-Trees

A B-tree is a multiway search tree designed to keep the tree shallow, especially for data stored externally on disks. The uploaded Splay Trees and B-Trees lecture gives the order-M properties below. ■filecite■turn12file1■L497-L505■

Properties from the lecture

1. The root is either a leaf or has between 2 and M children.
2. Each non-root, non-leaf node has between ceil(M/2) and M children.
3. All leaves are at the same depth.
4. Internal-node keys guide the search toward the appropriate child/subtree.
5. A node with k children has k–1 separator keys in the standard B-tree representation.

> Terminology warning: Some textbooks define 'order' differently. In these notes, M follows the uploaded lecture's definition: maximum number of children per node.

Order 3 example

For a B-tree of order M = 3, a node can have at most 3 children and therefore at most 2 keys. Non-root internal nodes require at least ceil(3/2) = 2 children. Search compares the target with the keys and descends into the corresponding child interval.

```
Example node:
       [  6 | 13  ]
       /     |     \
    <6    6..13    >13

Search 11: compare with 6, then 13; descend into middle child.
```

Search, insertion and splitting

Search: examine the node's separator keys; if the key is not present, choose the child interval containing it. **Insertion:** insert into the appropriate leaf; if the node exceeds capacity, split it and promote a separator key to the parent. A split can propagate upward and may create a new root. All leaves remain at the same depth.

Why B-trees are useful

Their high branching factor reduces tree height and therefore reduces disk accesses. They are commonly used in database and external-storage indexing contexts, matching the supplied lecture's motivation.

Structure	Children per node	Balancing approach
BST	At most 2	No automatic balancing rule in an ordinary BST.
AVL	At most 2	Strict height balance using rotations.
Splay	At most 2	Self-adjustment after access.
B-tree	Many children (up to M)	Splits/promotions maintain multiway search-tree properties.
Red-black tree	At most 2	Colour rules maintain logarithmic height.

5. Red-Black Trees

A red-black tree is a BST with a colour (red or black) stored at each node. The supplied slides list five properties: each node is red or black; the root is black; NIL leaves are black; a red node has black children; and every path from a node to descendant leaves contains the same number of black nodes. ■filecite■turn12file2■L35-L43■

The five properties

1. Every node is either RED or BLACK.
2. The root is BLACK.
3. Every NIL leaf/sentinel is BLACK.
4. If a node is RED, both its children are BLACK (no red-red parent-child pair).
5. Every path from a node to descendant NIL leaves has the same black-node count.

Why the rules matter

These colour constraints prevent the tree from becoming too tall. The supplied slide states that a red-black tree with n internal nodes has height at most 2 log■(n+1), so search and standard BST operations take O(log n) time. ■filecite■turn12file2■L54-L68■

Rotations and insertion fix-up

Left and right rotations rearrange local parent-child links while preserving BST key order. After inserting a new red node, violations can include a red root or a red node whose parent is also red. The lecture groups insertion repair by the uncle's colour and the node's position: a red uncle leads to recolouring; a black uncle leads to rotations and recolouring, with mirror-image cases.

Aspect	Red-black tree	AVL tree
Balance rule	Colour constraints and equal black-height	Each node BF is –1, 0 or +1
Rebalancing	Recolouring and rotations	Single or double rotations
Height guarantee	O(log n)	O(log n)
Typical emphasis	Good overall update/search balance	More tightly height-balanced

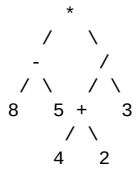
Exam diagram suggestion: draw a small BST with RED/BLACK labels, then explain the red-parent/red-child violation and the role of the uncle. Keep NIL leaves black.

6. Binary Expression Trees

A binary expression tree represents an arithmetic or Boolean expression. In the supplied slides, each leaf contains an operand, each non-leaf node contains a binary operator, and the operator is applied after evaluating its left and right subtrees. ■filecite■turn12file4■L42-L48■

Example

```
Expression: (8 - 5) * ((4 + 2) / 3)
```



The root operator is evaluated last; deeper subexpressions are evaluated first. The tree structure encodes precedence without requiring parentheses at every step. The lecture explicitly illustrates infix, prefix and postfix generation from the same tree. ■filecite■turn12file4■L100-L105■

Three traversals and expression notation

Traversal	Visit order	Notation produced
Inorder	Left → Root → Right	Infix (add parentheses for clarity)
Preorder	Root → Left → Right	Prefix / Polish notation
Postorder	Left → Right → Root	Postfix / Reverse Polish notation

Notation	Example for (4 + 2) × 3
Infix	(4 + 2) * 3
Prefix	* + 4 2 3
Postfix	4 2 + 3 *

Evaluating an expression tree

Recursively evaluate the left subtree and right subtree. If the current node is an operand, return its value. If it is an operator, apply that operator to the two returned values. The uploaded slides show this recursive evaluation pattern for +, –, × and ÷. ■filecite■turn12file4■L470-L484■

> Memory trick: Infix = operator in between; Prefix = operator before operands; Postfix = operator after operands.

7. Hashing Techniques

Hashing maps a key to an index in a hash table using a hash function. The uploaded hashing slides motivate it for frequent, time-critical lookups and state the goal of near-constant O(1) lookup time. They also note that a good hash function should be easy to compute and avoid collisions as much as possible. ■filecite■turn12file6■L16-L25■ ■filecite■turn12file6■L134-L143■

Hash function and collision

```
h(key) = key mod table_size

Example: table_size = 7
h(24) = 24 mod 7 = 3  -> slot 3

Collision: two different keys map to the same slot.
```

The hashing lecture demonstrates a character-sum function and explains why an order-insensitive sum can be poor: anagrams such as abc, cba and bca can map to the same location. ■filecite■turn12file6■L109-L129■

Collision resolution

Technique	How it works	Strength / limitation
Separate chaining	Each slot points to a collection/list of keys mapping there.	Simple; chains may grow if many collisions occur.
Linear probing	Try the next slot until an empty slot is found.	Simple but can create primary clustering.
Quadratic probing	Use quadratic offsets between probes.	Can reduce primary clustering; sequence and table size matter.
Double hashing	Use a second hash function to choose probe step.	Often spreads probes better; step must cover table slots.

Linear probing example

Let table size N = 7 and h(k) = k mod 7. Insert 10, 17, 24 in that order.

Key	Home slot	Resolution
10	3	Place at 3.
17	3	Slot 3 occupied; place at 4.
24	3	Slots 3 and 4 occupied; place at 5.

Load factor and complexity

Load factor λ = number of stored keys / number of table slots. A higher load factor generally increases collisions and probe lengths. Hash-table search/insert can be O(1) on average with a suitable hash function and controlled load factor; worst-case performance can be O(n).

> The source mentions prime table sizes as one way to improve distribution. Do not claim a hash function guarantees zero collisions; collisions are possible whenever keys map into a limited table.

8. Comparison sheet — all major Module 4 trees

Tree	Balancing method	Search / height takeaway	Key exam phrase
Ordinary BST	No enforced balance	O(h); worst case O(n)	Left < root < right
AVL	Balance factor + rotations	O(log n) height	BF = hL – hR; allowed –1, 0, +1
Splay	Splay accessed node to root	Amortized O(log n)	Zig, Zig-Zig, Zig-Zag
B-tree	Multiway nodes; split/promote keys	Shallow tree; all leaves same depth	Order M, child/key capacity
Red-black	Red/black colouring rules	Height O(log n)	Five properties; recolouring + rotations

9. Exam-ready questions

1. Define an AVL tree. Explain balance factor and all four rotation cases with diagrams.
2. Construct an AVL tree by inserting a given sequence of keys; show each rotation.
3. What is a splay tree? Explain Zig, Zig-Zig and Zig-Zag with diagrams.
4. Explain splay-tree insertion and deletion.
5. Define a B-tree of order M. State its properties and explain search and node splitting.
6. State the five red-black tree properties. Explain how insertion violations are repaired.
7. Compare AVL, splay, B-tree and red-black trees.
8. Construct a binary expression tree for a given expression and write its infix, prefix and postfix forms.
9. Explain hashing, hash functions, collisions and collision-resolution techniques.
10. Insert a sequence of keys into a hash table using linear probing and show the final table.
11. Explain tree terminology: root, edge, parent, child, sibling, degree, depth, height and leaf.

10. Last-minute memory sheet

Topic	Remember these words
AVL	BF = hL – hR; LL right; RR left; LR left-right; RL right-left.
Splay	Accessed node goes to root; Zig / Zig-Zig / Zig-Zag; locality.
B-tree	Multiway search tree; order M; split and promote; leaves same depth.
Red-black	Node colours; black root/NIL; no red-red edge; equal black-height.
Expression tree	Leaves are operands; internal nodes operators; inorder/prefix/postfix.
Hashing	Key → index; collision; chaining; linear/quadratic probing; double hashing; load factor.

Source note: These notes follow the uploaded slides and PDFs. The sorting-algorithms supplement is intentionally separate from this Module 4 document.